

Dokumentacja użytkownika końcowego PySQL

1. Wprowadzenie	2
2. Instalacja krok po kroku	2
3. Składnia i zasady języka PySQL	3
3.1. Deklaracja zmiennych i typów	3
3.2. Operatory	3
3.3. Pętle	3
3.4. Warunki	4
3.5. Funkcje	4
3.6. Operacje na tablicach	5
3.7. Importowanie funkcji	5
3.8. Dostęp do nadrzędnego scope'a	5

1. Wprowadzenie

PySQL to nowoczesny język programowania inspirowany składnią SQL, ale wzbogacony o imperatywne struktury, takie jak pętle, warunki, funkcje oraz system scope'ów. Został zaprojektowany z myślą o elastyczności – użytkownicy mogą manipulować danymi, wykonywać operacje matematyczne, a także korzystać z zaawansowanego systemu zarządzania zmiennymi.

PySQL łączy dwie filozofie:

- **Deklaratywną** (bazującą na SQL), gdzie użytkownik określa, co chce osiągnąć.
- **Imperatywną** (typową dla języków jak np. Python), gdzie użytkownik precyzyjnie określa kroki wykonania kodu.

Dzięki temu PySQL jest czytelny i intuicyjny, zarówno dla programistów baz danych, jak i dla osób przyzwyczajonych do języków ogólnego przeznaczenia. Główne cechy języka:

- możliwość wykonywania zapytań w stylu SQL
- system zmiennych i scope'ów
- wsparcie dla pętli
- typy danych
- obsługa funkcji
- obsługa błędów

2. Instalacja krok po kroku

- 2.1. Pobierz repozytorium PySQL
- 2.2. Zainstaluj potrzebne pakiety na środowisku lokalnym (wymagane są: antlr4-python3-runtime==4.13.0 oraz Python wersja co najmniej 3.8 lub nowszy)
- 2.3. Wygeneruj potrzebne pliki do obsługi języka poleceniem:
“ java -jar antlr-4.13.1-complete.jar -Dlanguage=Python3 -visitor PySQL.g4”
- 2.4. Uruchom przykładowy skrypt poleceniem:
“python3 PySQLInterpreter.py nazwa_pliku.txt”

3. Składnia i zasady języka PySQL

3.1. Deklaracja zmiennych i typów

Zmienne mogą być typu: int, float, string, bool, mixed bądź array<typ>

```
int x = 5
float y = 3.14
string s = "Hello"
bool flag = true
array<int> numbers = [1, 2, 3]
array<mixed> arr = ["Janek", 21]
```

Możliwa jest również konwersja typów

```
int j = 5
print((float)j)
// Konwersja bool na int
print((int)(j > 2))
```

3.2 Dostęp do elementów tablicy

W języku PySQL dostęp do poszczególnych elementów tablicy uzyskujemy stosując indeksację zaczynającą się od zera.

```
array<int> tablica = [0, 1, 2, 3, 4]
print(tablica[0])
```

3.3. Operatory

Język wspiera następujące operatory arytmetyczne i logiczne:

Operator	Opis	Przykład
+, -	Dodawanie, odejmowanie	x + y
*, /	Mnożenie, dzielenie	x/y
++, —	Inkrementacja, dekrementacja	x++
==, !=, <, <=, >, >=	Operatory porównania	x == y

3.4. Pętle

Język umożliwia iteracje po danych pętlami `for` oraz `while`, w następujący sposób:

```
for (int i = 3; i <= 5; i = i+1) do {
    print(i)
}

int j = 0
while (j < 5) do {
    print(j)
    j = j + 1
}

int j = 0
while (j < 5) do {
    j = j + 1
    if (j == 2) then continue
    if (j == 4) then break
    print(j)
}
```

3.5. Warunki

Język wspiera również instrukcje warunkowe, których przykładowe użycie zostało zaprezentowane poniżej:

```
if (x > 0) then print("x jest dodatnie") else print("x jest ujemne")
```

3.6. Funkcje

Kolejnym atutem języka jest możliwość deklarowania i wywoływania funkcji. Jak na przykładzie poniżej:

```
func add(a: int, b: int) -> int exec(
    return a + b
)
print(add(2, 5))
```

Tak poza tym mamy dostępne jeszcze funkcje wbudowane, które operują na łańcuchach znaków, tj. toUpper, toLower, startsWith, endsWith, contains

```
string greeting = "Hello, PySQL!"
#zamiana małych liter na wielkie
print(toUpper(greeting)) // "HELLO, PYTHON!"
#zamiana wielkich liter na małe
print(toLower(greeting)) // "hello, pysql!"
#Porównanie czy początkowy ciąg znaków jest równy podanemu przez użytkownika
print(startsWith(greeting, "Hello")) // true
#Porównanie czy końcowy ciąg znaków jest równy podanemu przez użytkownika
print(endsWith(greeting, "SQL!")) // true
#Sprawdzenie czy dany znak bądź łańcuch jest zawartością stringa
print(contains(greeting, "Py")) // true
```

Oprócz tego język daje nam szerokie możliwości wykorzystywania funkcji agregujących na tablicach. Oto przykład:

```
array<mixed> data = [10, 20, 30, 40, 50]
#Liczba elementów
print(len(data)) // 5
#Suma wszystkich elementów
print(sum(data)) // 150
#Średnia wszystkich elementów
print(avg(data)) // 30.0
#Wartość minimalna
print(min(data)) // 10
#Wartość maksymalna
print(max(data)) // 50
#Wartość mediany
print(median(data)) // 30.0
```

3.7. Operacje SQL-like

Elementem charakterystycznym naszego języka jest możliwość tworzenia zapytań, które modyfikują bądź wyświetlają na dane z tabeli. Metoda tworzenia zapytań w PySQL została przedstawiona poniżej.

```
array<float> prices = [9.99, 5.49, 12.99, 3.25]
//wyświetlanie danych z tabeli
print(SELECT _ FROM prices)
//filtracja danych
print(SELECT _ FROM prices WHERE _ > 5.0)
//Manipulacja wartościami – operacje arytmetyczne
print(SELECT _ * 1.2 FROM prices)
//Funkcje agregujące w zapytaniach
array<int> numbers = [1, 2, 3, 4 5]
print(SELECT sum(_) FROM numbers)
//Konwersja typów i łączenie wartości
string currency = "pln"
print(SELECT toUpper((string)_ + currency) FROM prices WHERE _ > 5.0)
//Zagnieżdżone zapytania SELECT
array<array<int>> matrix = [[1,2], [3,4], [5,6]]
print(SELECT SELECT _ * 2 FROM _ FROM matrix)
//Własne funkcje w SELECT
func square(n: int) -> int exec (
    return n * n
)
print(SELECT square(_) FROM numbers)
```

3.8. Importowanie funkcji

Po słowie `from` podajemy ścieżkę do pliku, a po słowie `import` określamy które funkcje chcielibyśmy wykorzystać.

```
from "utils.txt" import add, divide
```

3.9. Dostęp do nadrzędnego scope'a

W PySQL mamy możliwość tworzenia zakresów, w obrębie których definiujemy zmienne. Wyrażeniem, które daje nam dostęp do wartości nadrzędnych jest `parent::x`

```
int x = 1      // Poziom 0 (globalny)
{
    int x = 10 // Poziom 1 (nadrzędny)
    {
        int x = 100 // Poziom 2 (bieżący)

        // Wyświetlamy wartości z 3 różnych poziomów za jednym razem.
        print(x, parent::x, parent::parent::x)
    }
}
```

3.10 Komentarze

```
/\
komentarz blokowy
(...)
/\
//komentarz zwykły
```

3.11 Error Handling

Dodatkowo język posiada silny system diagnostyki błędów typu, semantycznych oraz składniowych. Ponadto Wskazuje konkretną linię błędu, co ułatwia debugowanie

```
// Błąd: niezdefiniowana zmienna
print(x)
// PySQLNameError: Undefined variable 'x' at line 1:6"
)

// Błąd: dzielenie przez zero
print(5 / 0)
// An unexpected error occurred: Division by zero at line 1
)

// Błąd: nieprawidłowy typ w operacji
print(5 + "hello")
// An unexpected error occurred: Incompatible types for '+' at line 1

// Błąd: nieprawidłowa deklaracja zmiennej
int 1x = 5
print(1x)
//An unexpected error occurred: An unexpected error occurred: Invalid variable name '1x' at line 1

// Błąd: nieprawidłowa deklaracja funkcji
unc 1func() -> int exec (return 5)
//PySQLNameError: Invalid function name at line 1 (near '1func')

// Błąd: nieprawidłowy typ zwracany z funkcji
func invalidReturn() -> int exec (
    return true
)
print(invalidReturn())
//An unexpected error occurred: Function 'invalidReturn' should return int, got bool. Called at line 4

// Błąd: nieprawidłowa liczba argumentów w funkcji
func add(a:int) -> int exec (
    return a + 1
)
print(add(5, 10))
// An unexpected error occurred: Function 'add' expects 1 args, got 2 at line 4

// Błąd: nieprawidłowa składnia w pętli
for (i = 0; i < 5; i = i + 1) print(i)
// PySQLSyntaxError: Unexpected token 'print' (type 44); expecting a specific token or sequence at line 1 (near 'print')

// Błąd: nieprawidłowa składnia w warunku if
if (true) print("OK")
// PySQLSyntaxError: missing 'then' at 'print' at line 1 (near 'print')
```